Recursion

1. What Is Recursion?

2. The Three Laws of Recursion

3. Tower of Hanoi

4. Exploring a Maze

5.2 What Is Recursion?

Recursion is a method of solving problems that involves **breaking a problem down into smaller and smaller subproblems** until you get to a small enough problem that it can be solved trivially.

Recursion is a method of solving problems that involves **breaking a problem down into smaller and smaller subproblems** until you get to a small enough problem that it can be solved trivially.

Usually recursion involves a function calling itself. It allows us to write elegant solutions to problems that may otherwise be very difficult to program!

5.3 Calculating the Sum of a Vector of Numbers

Suppose that you want to calculate the sum of a vector of numbers such as: `{1, 3, 5, 7, 9}`. An iterative function that computes the sum is shown below. The function uses an accumulator variable (`theSum`) to compute a running total by starting with 0 and adding each number in the vector.

Suppose that you want to calculate the sum of a vector of numbers such as: `{1, 3, 5, 7, 9}`. An iterative function that computes the sum is shown below. The function uses an accumulator variable (`theSum`) to compute a running total by starting with 0 and adding each number in the vector.

In [2]:
```
#include <iostream>
#include <vector>
using namespace std;

int listSum(const vector<int>& numList) {
    int theSum = 0;
    for (int value : numList) {
        theSum += value;
    }
    return theSum;
}

int main() {
    cout << listSum({1, 3, 5, 7, 9}) << endl;
    cout << listSum({}) << endl;   // the empty vector sums to 0
    return 0;
}
```

```
25
0
```

Without loops, how would you compute the sum of a vector of numbers?

Without loops, how would you compute the sum of a vector of numbers?

You might start by recalling that addition is a function that is defined for two parameters, a pair of numbers. To redefine the problem from adding a vector to adding pairs of numbers, we could rewrite the vector as a **fully parenthesized expression**. Such an expression looks like this:

Without loops, how would you compute the sum of a vector of numbers?

You might start by recalling that addition is a function that is defined for two parameters, a pair of numbers. To redefine the problem from adding a vector to adding pairs of numbers, we could rewrite the vector as a **fully parenthesized expression**. Such an expression looks like this:

$$(1 + (3 + (5 + (7 + 9))))$$

Notice that the innermost set of parentheses, , is a problem that we can solve without a loop or any special constructs! In fact, we can use the following sequence of simplifications to compute a final sum:

Notice that the innermost set of parentheses, , is a problem that we can solve without a loop or any special constructs! In fact, we can use the following sequence of simplifications to compute a final sum:

$$
\begin{aligned}
total = \ & (1 + (3 + (5 + (7 + 9)))) \\
total = \ & (1 + (3 + (5 + 16))) \\
total = \ & (1 + (3 + 21)) \\
total = \ & (1 + 24) \\
total = \ & 25
\end{aligned}
$$

First, restate the sum problem for a C++ `vector`. The sum starting at index `i` is `numList[i]` plus the sum starting at `i + 1`. When `i == numList.size()`, the remaining range is empty and its sum is 0. Passing one vector plus an index avoids allocating a new sub-vector at every call.

First, restate the sum problem for a C++ `vector`. The sum starting at index `i` is `numList[i]` plus the sum starting at `i + 1`. When `i == numList.size()`, the remaining range is empty and its sum is 0. Passing one vector plus an index avoids allocating a new sub-vector at every call.

$$\texttt{listSumFrom(numList, i)} = \begin{cases} \texttt{numList[i] + listSumFrom(numList, i + 1)} & \text{if i < numList.size()} \\ \texttt{0} & \text{if i == numList.size()} \end{cases}$$

In [3]:

```cpp
#include <cstddef>
#include <iostream>
#include <vector>
using namespace std;

int listSumFrom(const vector<int>& numList, size_t index) {
    if (index == numList.size()) {
        return 0;                                   // empty remaining range
    }
    return numList[index] + listSumFrom(numList, index + 1);
}

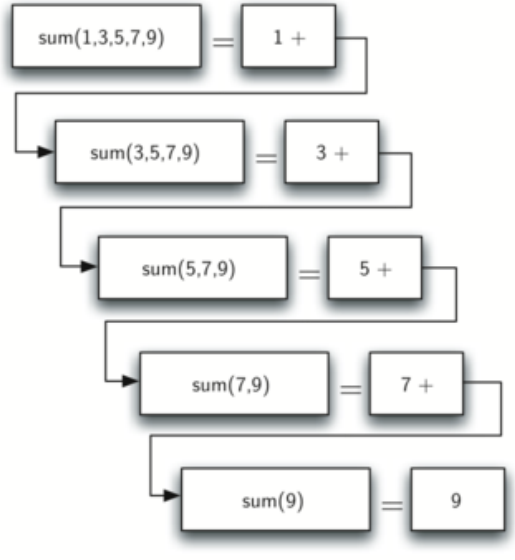
int listSum(const vector<int>& numList) {
    return listSumFrom(numList, 0);
}

int main() {
    cout << listSum({1, 3, 5, 7, 9}) << endl;
    cout << listSum({}) << endl;                    // safe: prints 0
    return 0;
}
```

```
25
0
```

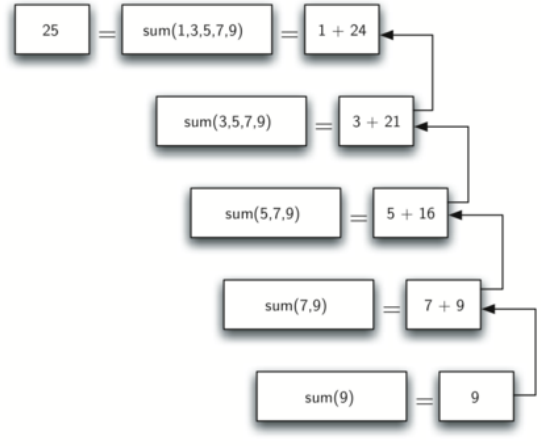
Figure below shows the series of recursive calls that are needed to sum the vector. You should think of this series of calls as a series of simplifications.

Figure below shows the series of recursive calls that are needed to sum the vector. You should think of this series of calls as a series of simplifications.



When we reach the point where the problem is as simple as it can get, we begin to piece together the solutions of each of the small problems until the initial problem is solved!

When we reach the point where the problem is as simple as it can get, we begin to piece together the solutions of each of the small problems until the initial problem is solved!



5.4 The Three Laws of Recursion

Like robots in Asimov's stories, all recursive algorithms must obey three important laws:

1. A recursive algorithm must have a base case.

Like robots in Asimov's stories, all recursive algorithms must obey three important laws:

1. A recursive algorithm must have a base case.

2. A recursive algorithm must change its state and move toward the base case.

Like robots in Asimov's stories, all recursive algorithms must obey three important laws:

1. A recursive algorithm must have a base case.

2. A recursive algorithm must change its state and move toward the base case.

3. A recursive algorithm must call itself recursively.

A base case is a problem small enough to solve directly. In `listSumFrom()`, the base case is an empty remaining range (`index == numList.size()`), whose sum is 0.

A base case is a problem small enough to solve directly. In `listSumFrom()`, the base case is an empty remaining range (`index == numList.size()`), whose sum is 0.

A change of state means moving toward the base case. `listSumFrom()` keeps the vector fixed and advances `index`, so the unprocessed range becomes shorter on every recursive call.

A base case is a problem small enough to solve directly. In `listSumFrom()`, the base case is an empty remaining range (`index == numList.size()`), whose sum is 0.

A change of state means moving toward the base case. `listSumFrom()` keeps the vector fixed and advances `index`, so the unprocessed range becomes shorter on every recursive call.

The final law is that the algorithm must call itself. This is the very definition of recursion.

As a novice programmer, you have learned that functions are good because you can take a large problem and break it up into smaller problems. The smaller problems can be solved by writing a function to solve each problem.

As a novice programmer, you have learned that functions are good because you can take a large problem and break it up into smaller problems. The smaller problems can be solved by writing a function to solve each problem.

When we talk about recursion it may seem that we are talking ourselves in circles. We have a problem to solve with a function, but that function solves the problem by calling itself! We are solving a problem by breaking it down into a smaller and easier problems.

In [4]:
```
display_quiz(path+"recursive1.json", max_width=800)
```

How many recursive calls are made when computing the sum of the vector {2, 4, 6, 8, 10} with listSum()?

- ○ 4
- ○ 6
- ○ 5
- ○ 3

In [5]:
```
display_quiz(path+"recursive2.json", max_width=800)
```

Suppose you are going to write a recursive function to calculate the factorial of a number. fact(n) returns n * (n-1) * (n-2) * ... where the factorial of zero is defined to be 1. What would be the most efficient and appropriate base case?

- ○ `n >= 0`
- ○ `n <= 1`
- ○ `n == 0`
- ○ `n == 1`

5.5 Converting an Integer to a String in Any Base

Suppose you want to convert an integer to a string in some base between binary and hexadecimal. For example, convert the integer 10 to its string representation in decimal as "10", or to its string representation in binary as "1010".

Suppose you want to convert an integer to a string in some base between binary and hexadecimal. For example, convert the integer 10 to its string representation in decimal as "10", or to its string representation in binary as "1010".

While there are many algorithms to solve this problem, including the algorithm discussed in the stack section, the recursive formulation of the problem is very elegant.

Suppose you want to convert an integer to a string in some base between binary and hexadecimal. For example, convert the integer 10 to its string representation in decimal as "10", or to its string representation in binary as "1010".

While there are many algorithms to solve this problem, including the algorithm discussed in the stack section, the recursive formulation of the problem is very elegant.

Let's look at a concrete example using base `10` and the number `769`. Suppose we have a sequence of characters corresponding to the first 10 digits, like `convertString = "0123456789"`. It is easy to convert a number less than 10 to its string equivalent by looking it up in the sequence.

If we can arrange to break up the number `769` into three single-digit numbers, `7`, `6`, and `9`, then converting it to a string is simple. A number less than 10 sounds like a good base case.

If we can arrange to break up the number `769` into three single-digit numbers, `7`, `6`, and `9`, then converting it to a string is simple. A number less than 10 sounds like a good base case.

Knowing what our base is suggests that the overall algorithm will involve three components:

If we can arrange to break up the number `769` into three single-digit numbers, `7`, `6`, and `9`, then converting it to a string is simple. A number less than 10 sounds like a good base case.

Knowing what our base is suggests that the overall algorithm will involve three components:

1. Reduce the original number to a series of single-digit numbers.
2. Convert the single digit-number to a string using a lookup.
3. Concatenate the single-digit strings together to form the final result.

The next step is to figure out how to change state and make progress toward the base case.

The next step is to figure out how to change state and make progress toward the base case.

Let's consider what mathematical operations might reduce a number. The most likely candidates are division and subtraction. While subtraction might work, it is unclear what we should subtract from what. Integer division with remainders gives us a clear direction!

The next step is to figure out how to change state and make progress toward the base case.

Let's consider what mathematical operations might reduce a number. The most likely candidates are division and subtraction. While subtraction might work, it is unclear what we should subtract from what. Integer division with remainders gives us a clear direction!

Using integer division to divide `769` by `10`, we get `76` with a remainder of `9`. This gives us two good results.

1. The remainder is a number less than our base that can be converted to a string immediately by lookup.

The next step is to figure out how to change state and make progress toward the base case.

Let's consider what mathematical operations might reduce a number. The most likely candidates are division and subtraction. While subtraction might work, it is unclear what we should subtract from what. Integer division with remainders gives us a clear direction!

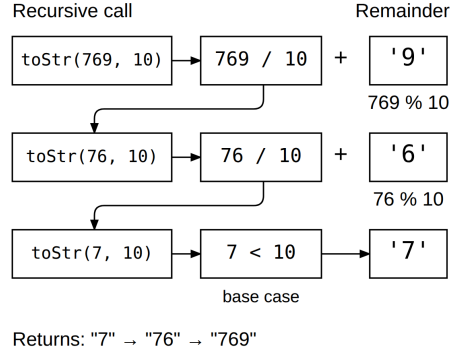
Using integer division to divide `769` by `10`, we get `76` with a remainder of `9`. This gives us two good results.

1. The remainder is a number less than our base that can be converted to a string immediately by lookup.

2. We get a number that is smaller than our original and moves us toward the base case of having a single number less than our base.

Now our job is to convert `76` to its string representation. Again we will use integer division plus remainder to get results of `7` and `6` respectively.

Now our job is to convert `76` to its string representation. Again we will use integer division plus remainder to get results of `7` and `6` respectively.



In [6]:

```cpp
#include <iostream>
#include <string>
using namespace std;

// The algorithm outlined above for any base between 2 and 16.
string toStr(int n, int base) {
    string convertString = "0123456789ABCDEF";
    if (n < base) {
        return string(1, convertString[n]);
    } else {
        return toStr(n / base, base) + convertString[n % base];
    }
}

int main() {
    cout << toStr(1453, 16) << endl;
    return 0;
}
```

```
5AD
```

In [6]:

```cpp
#include <iostream>
#include <string>
using namespace std;

// The algorithm outlined above for any base between 2 and 16.
string toStr(int n, int base) {
    string convertString = "0123456789ABCDEF";
    if (n < base) {
        return string(1, convertString[n]);
    } else {
        return toStr(n / base, base) + convertString[n % base];
    }
}

int main() {
    cout << toStr(1453, 16) << endl;
    return 0;
}
```

```
5AD
```

Notice that in line 8 we check for the base case where `n` is less than the base we are converting to. When we detect the base case, we stop recursing and simply return the string from the `convertString` sequence. In line 11 we satisfy both the second and third laws – by making the recursive call and by reducing the problem size–using division.

In [6]:

```cpp
#include <iostream>
#include <string>
using namespace std;

// The algorithm outlined above for any base between 2 and 16.
string toStr(int n, int base) {
    string convertString = "0123456789ABCDEF";
    if (n < base) {
        return string(1, convertString[n]);
    } else {
        return toStr(n / base, base) + convertString[n % base];
    }
}

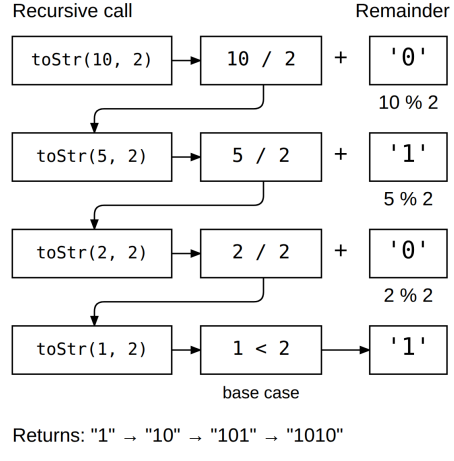
int main() {
    cout << toStr(1453, 16) << endl;
    return 0;
}
```

```
5AD
```

Notice that in line 8 we check for the base case where `n` is less than the base we are converting to. When we detect the base case, we stop recursing and simply return the string from the `convertString` sequence. In line 11 we satisfy both the second and third laws – by making the recursive call and by reducing the problem size–using division.

Let's trace the algorithm; this time we will convert the number `10` to its base 2 string representation (`"1010"`):

Let's trace the algorithm; this time we will convert the number `10` to its base 2 string representation (`"1010"`):



It looks like the digits are in the wrong order. The algorithm works correctly because we make the recursive call first on line 11, then we add the string representation of the remainder.

It looks like the digits are in the wrong order. The algorithm works correctly because we make the recursive call first on line 11, then we add the string representation of the remainder.

If we reversed returning the `convertString` lookup and returning the `toStr()` call, the resulting string would be backward! But by delaying the concatenation operation until after the recursive call has returned, we get the result in the proper order! This should remind you of our discussion of stacks back in the previous chapter.

Exercise: Write a function using recursion that takes a string as a parameter and returns a new string that is the reverse of the old string.

In [ ]:

```cpp
#include <cassert>
#include <string>
using namespace std;
string reverse(string s) {
    if (s.size() <= 1) {
        return ____;            // handles "" and one character
    }
    return ____ + s[0];
}
int main() {
    assert(reverse("") == "");
    assert(reverse("hello") == "olleh");
}
```

In [7]:
```
display_quiz(path+"reverse.json", max_width=800)
```

Which base case makes a recursive C++ reverse(string s) function correct for both an empty string and a one-character string?

- ○
```
if (s.size() <= 1) return s;
```
- ○
```
if (s.empty()) return s[0];
```
- ○
```
if (s.size() == 1) return s;
```
- ○
```
if (s.size() >= 1) return s;
```

5.6 Stack Frames: Implementing Recursion

Suppose that instead of concatenating the result of the recursive call to `toStr()` with the string from `convertString`, we modified our algorithm to push the strings onto a stack instead of making the recursive call:

Using the STL stack, we can replace recursion with an explicit stack of characters:

Using the STL stack, we can replace recursion with an explicit stack of characters:

In [8]:
```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

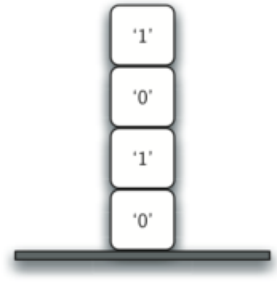
string toStr(int n, int base) {
    stack<char> rStack;
    string convertString = "0123456789ABCDEF";
    while (n > 0) {
        rStack.push(convertString[n % base]);
        n = n / base;
    }
    string res = "";
    while (!rStack.empty()) {
        res = res + rStack.top();
        rStack.pop();
    }
    return res;
}

int main() { cout << toStr(1453, 16) << endl; }
```

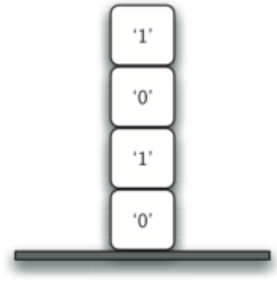
```
5AD
```

Each time we make a call to `toStr()`, we push a character on the stack. Returning to the previous example we can see that after the fourth call to `toStr()` the stack would look like Figure below:

Each time we make a call to `toStr()`, we push a character on the stack. Returning to the previous example we can see that after the fourth call to `toStr()` the stack would look like Figure below:



Each time we make a call to `toStr()`, we push a character on the stack. Returning to the previous example we can see that after the fourth call to `toStr()` the stack would look like Figure below:



Notice that now we can simply pop the characters off the stack and concatenate them into the final result, `"1010"`.

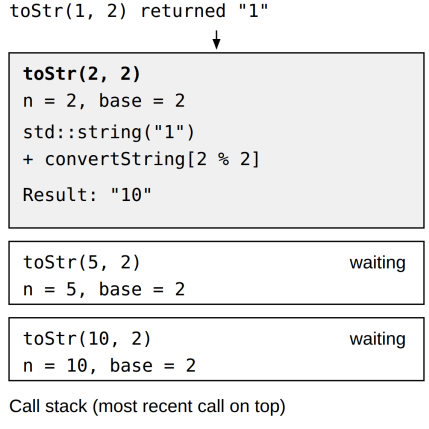
The previous example gives insight into how C++ implements recursive calls. For each active function call, the runtime maintains a stack frame containing the call's parameters, local variables, and return information.

The previous example gives insight into how C++ implements recursive calls. For each active function call, the runtime maintains a stack frame containing the call's parameters, local variables, and return information.

When the function returns, the return value is left on top of the stack for the calling function to access.

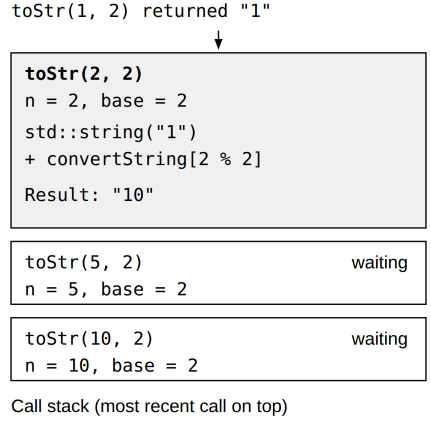
Figure below illustrates the call stack after the return statement:

Figure below illustrates the call stack after the return statement:



Call stack (most recent call on top)

Figure below illustrates the call stack after the return statement:



Call stack (most recent call on top)

The call `toStr(2 / 2, 2)` returns `"1"`. That value is substituted into `toStr(1, 2) + convertString[2 % 2]`, producing `"10"` before the earlier calls continue returning.

In this way, the `C++` call stack takes the place of the stack we used explicitly. In our vector summing example, you can think of the return value on the stack taking the place of an accumulator variable.

In this way, the `C++` call stack takes the place of the stack we used explicitly. In our vector summing example, you can think of the return value on the stack taking the place of an accumulator variable.

The stack frames also provide a scope for the variables used by the function. Even though we are calling the same function over and over, each call creates a new scope for the variables that are local to the function.

5.9 Complex Recursive Problems

5.10 Tower of Hanoi

The Tower of Hanoi puzzle was invented by the French mathematician Edouard Lucas in 1883. He was inspired by a legend that tells of a Hindu temple where the puzzle was presented to young priests.

The Tower of Hanoi puzzle was invented by the French mathematician Edouard Lucas in 1883. He was inspired by a legend that tells of a Hindu temple where the puzzle was presented to young priests.

At the beginning of time, the priests were given three poles and a stack of 64 gold disks, each disk a little smaller than the one beneath it. Their assignment was to transfer all 64 disks from one of the three poles to another, with two important constraints.

The Tower of Hanoi puzzle was invented by the French mathematician Edouard Lucas in 1883. He was inspired by a legend that tells of a Hindu temple where the puzzle was presented to young priests.

At the beginning of time, the priests were given three poles and a stack of 64 gold disks, each disk a little smaller than the one beneath it. Their assignment was to transfer all 64 disks from one of the three poles to another, with two important constraints.

1. They could only move one disk at a time.
2. They could never place a larger disk on top of a smaller one.

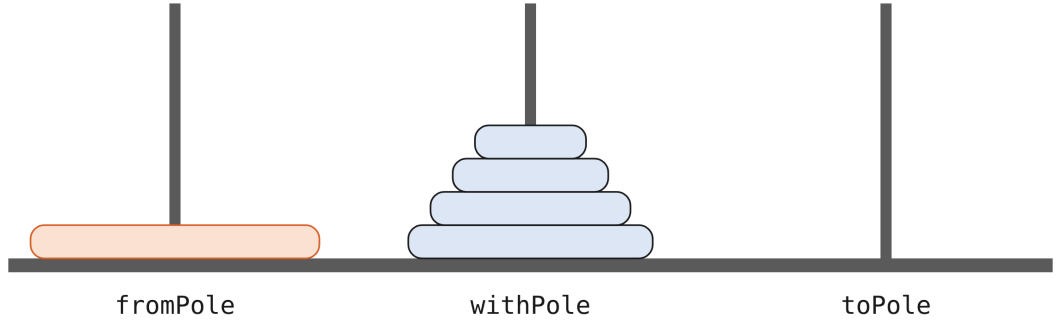
The priests worked very efficiently, day and night, moving one disk every second. When they finished their work, the legend said, the temple would crumble into dust and the world would vanish.

The priests worked very efficiently, day and night, moving one disk every second. When they finished their work, the legend said, the temple would crumble into dust and the world would vanish.

Although the legend is interesting, you need not worry about the world ending any time soon. The number of moves required to correctly move a tower of 64 disks is . At a rate of one move per second, that is years!

Figure below shows an example of a configuration of disks in the middle of a move from the first peg to the third. Notice that, as the rules specify, the disks on each peg are stacked so that smaller disks are always on top of the larger disks:

Figure below shows an example of a configuration of disks in the middle of a move from the first peg to the third. Notice that, as the rules specify, the disks on each peg are stacked so that smaller disks are always on top of the larger disks:



Let's think about this problem from the bottom up. Suppose you have a tower of five disks, originally on peg one. If you already knew how to move a tower of four disks to peg two, you could then easily move the bottom disk to peg three, and then move the tower of four from peg two to peg three.

Let's think about this problem from the bottom up. Suppose you have a tower of five disks, originally on peg one. If you already knew how to move a tower of four disks to peg two, you could then easily move the bottom disk to peg three, and then move the tower of four from peg two to peg three.

But what if you do not know how to move a tower of height four? Suppose that you knew how to move a tower of height three to peg three; then it would be easy to move the fourth disk to peg two and move the three from peg three on top of it.

Let's think about this problem from the bottom up. Suppose you have a tower of five disks, originally on peg one. If you already knew how to move a tower of four disks to peg two, you could then easily move the bottom disk to peg three, and then move the tower of four from peg two to peg three.

But what if you do not know how to move a tower of height four? Suppose that you knew how to move a tower of height three to peg three; then it would be easy to move the fourth disk to peg two and move the three from peg three on top of it.

But what if you still do not know how to do this? Surely you would agree that moving a single disk to peg three is easy enough, trivial you might even say. This sounds like a base case in the making.

Here is a high-level outline of how to move a tower of height from the starting pole to the goal pole, using an intermediate pole:

Here is a high-level outline of how to move a tower of height from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height from the starting pole to an intermediate pole via the goal pole.

Here is a high-level outline of how to move a tower of height from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height from the starting pole to an intermediate pole via the goal pole.

2. Move the remaining disk from the starting pole to the final pole.

Here is a high-level outline of how to move a tower of height from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height from the starting pole to an intermediate pole via the goal pole.

2. Move the remaining disk from the starting pole to the final pole.

3. Move the tower of height from the intermediate pole to the goal pole via the starting pole.

Here is a high-level outline of how to move a tower of height from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height from the starting pole to an intermediate pole via the goal pole.

2. Move the remaining disk from the starting pole to the final pole.

3. Move the tower of height from the intermediate pole to the goal pole via the starting pole.

The simplest non-empty Tower of Hanoi problem is a tower of one disk: we move that single disk to its destination. In the code, the base case is a tower of height 0, where there is nothing to move. In addition, the steps outlined above move us toward the base case by reducing the height of the tower in steps 1 and 3.

In [10]:

```cpp
#include <iostream>
#include <string>
using namespace std;

void moveDisk(string fromP, string toP) {
    cout << "moving disk from " << fromP << " to " << toP << endl;
}
void moveTower(int height, string fromPole, string toPole, string withPole) {
    if (height >= 1) {
        moveTower(height - 1, fromPole, withPole, toPole);
        moveDisk(fromPole, toPole);
        moveTower(height - 1, withPole, toPole, fromPole);
    }
}
int main() { moveTower(3, "A", "B", "C"); }
```

```
moving disk from A to B
moving disk from A to C
moving disk from B to C
moving disk from A to B
moving disk from C to A
moving disk from C to B
moving disk from A to B
```

The base case is the tower of height 0; in this case there is nothing to do, so the `moveTower()` function returns. The important thing to remember about handling the base case this way is that simply returning from `moveTower()` is what finally allows the `moveDisk()` function to be called.

The base case is the tower of height 0; in this case there is nothing to do, so the `moveTower()` function returns. The important thing to remember about handling the base case this way is that simply returning from `moveTower()` is what finally allows the `moveDisk()` function to be called.

Now that you have seen the code for both `moveTower()` and `moveDisk()`, you may be wondering why we do not have a data structure that explicitly keeps track of what disks are on what poles.

The base case is the tower of height 0; in this case there is nothing to do, so the `moveTower()` function returns. The important thing to remember about handling the base case this way is that simply returning from `moveTower()` is what finally allows the `moveDisk()` function to be called.

Now that you have seen the code for both `moveTower()` and `moveDisk()`, you may be wondering why we do not have a data structure that explicitly keeps track of what disks are on what poles.

If you explicitly tracked the disks, you could use three `Stack` objects, one per pole. The recursive C++ program instead relies on the runtime call stack to remember the suspended calls.

5.11 Exploring a Maze

In this section we will look at a problem that has relevance to the expanding world of robotics: how do you find your way out of a maze?

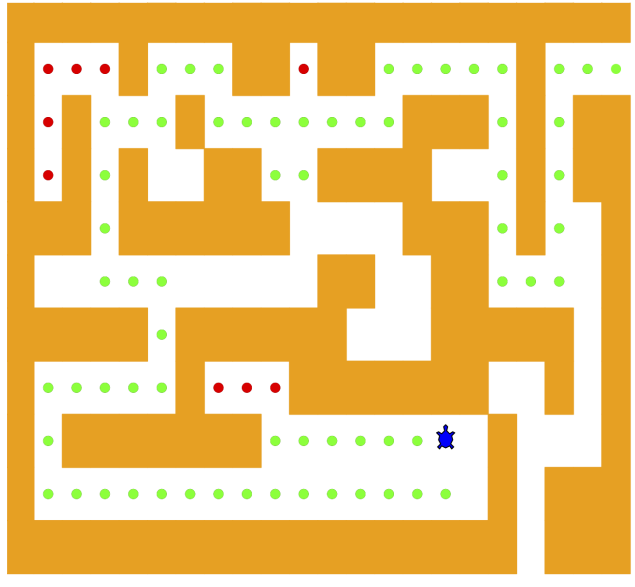
In this section we will look at a problem that has relevance to the expanding world of robotics: how do you find your way out of a maze?

The problem we want to solve is to help our turtle find its way out of a virtual maze. The maze problem has roots as deep as the Greek myth about Theseus, who was sent into a maze to kill the Minotaur.

In this section we will look at a problem that has relevance to the expanding world of robotics: how do you find your way out of a maze?

The problem we want to solve is to help our turtle find its way out of a virtual maze. The maze problem has roots as deep as the Greek myth about Theseus, who was sent into a maze to kill the Minotaur.

Theseus used a ball of thread to help him find his way back out again once he had finished off the beast. In our problem we will assume that our turtle is dropped down somewhere into the middle of the maze and must find its way out.



To make it easier for us we will assume that our maze is divided up into squares. Each square of the maze is either open or occupied by a section of wall. The turtle can only pass through the open squares of the maze. If the turtle bumps into a wall, it must try a different direction. Here is the procedure:

To make it easier for us we will assume that our maze is divided up into squares. Each square of the maze is either open or occupied by a section of wall. The turtle can only pass through the open squares of the maze. If the turtle bumps into a wall, it must try a different direction. Here is the procedure:

1. From our starting position we will first try going north one square and then **recursively try our procedure from there**.

To make it easier for us we will assume that our maze is divided up into squares. Each square of the maze is either open or occupied by a section of wall. The turtle can only pass through the open squares of the maze. If the turtle bumps into a wall, it must try a different direction. Here is the procedure:

1. From our starting position we will first try going north one square and then **recursively try our procedure from there**.

2. If we are not successful by trying a northern path as the first step then we will take a step to the south and recursively repeat our procedure.

3. If south does not work then we will try a step to the west as our first step and recursively apply our procedure.

3. If south does not work then we will try a step to the west as our first step and recursively apply our procedure.

4. If north, south, and west have not been successful then we will apply the procedure recursively from a position one step to our east.

3. If south does not work then we will try a step to the west as our first step and recursively apply our procedure.

4. If north, south, and west have not been successful then we will apply the procedure recursively from a position one step to our east.

5. If none of these directions works then there is no way to get out of the maze and we fail.

Now that sounds easy, but there are a couple of details to talk about! Suppose we take our first recursive step by going north. By following our procedure, our next step would also be to the north. But if the north is blocked by a wall, we must look at the next step of the procedure and try going to the south.

Now that sounds easy, but there are a couple of details to talk about! Suppose we take our first recursive step by going north. By following our procedure, our next step would also be to the north. But if the north is blocked by a wall, we must look at the next step of the procedure and try going to the south.

Unfortunately, that step to the south brings us right back to our original starting place. If we apply the recursive procedure from there, we will just go back one step to the North and be in an infinite loop! So we must have a strategy to remember where we have been!

Now that sounds easy, but there are a couple of details to talk about! Suppose we take our first recursive step by going north. By following our procedure, our next step would also be to the north. But if the north is blocked by a wall, we must look at the next step of the procedure and try going to the south.

Unfortunately, that step to the south brings us right back to our original starting place. If we apply the recursive procedure from there, we will just go back one step to the North and be in an infinite loop! So we must have a strategy to remember where we have been!

In this case we will assume that we have a **bag of bread crumbs we can drop along our way.** If we take a step in a certain direction and find that there is a bread crumb already on that square, we know that we should immediately back up and try the next direction in our procedure. As we will see when we look at the code for this algorithm, **backing up is as simple as returning from a recursive function call.**

As we do for all recursive algorithms, let us review the base cases. Some of them you may already have guessed based on the description in the previous paragraph. In this algorithm, there are four base cases to consider:

As we do for all recursive algorithms, let us review the base cases. Some of them you may already have guessed based on the description in the previous paragraph. In this algorithm, there are four base cases to consider:

1. The turtle has run into a wall. Since the square is occupied by a wall, no further exploration can take place.

2. The turtle has found a square that has already been explored. We do not want to continue exploring from this position so we don't get into a loop.

As we do for all recursive algorithms, let us review the base cases. Some of them you may already have guessed based on the description in the previous paragraph. In this algorithm, there are four base cases to consider:

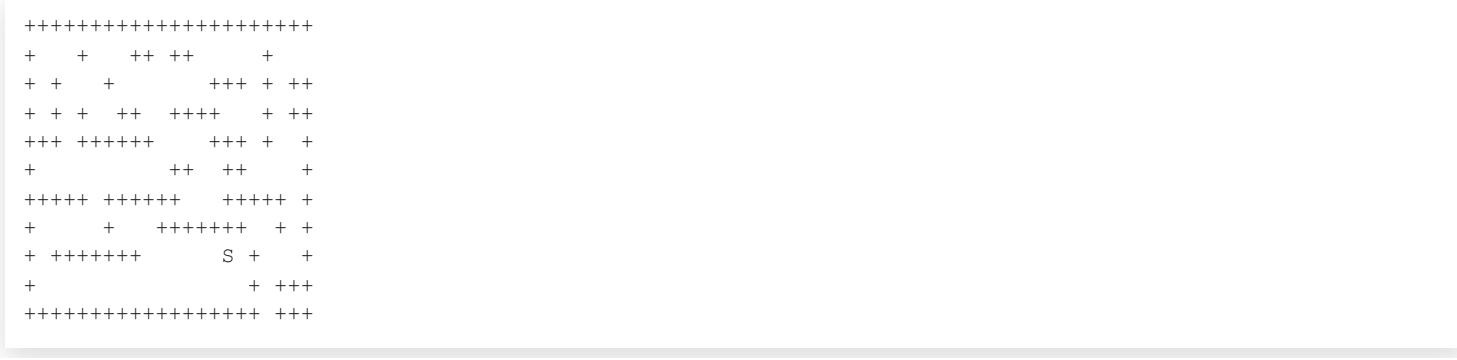
1. The turtle has run into a wall. Since the square is occupied by a wall, no further exploration can take place.

2. The turtle has found a square that has already been explored. We do not want to continue exploring from this position so we don't get into a loop.

3. We have found an outside edge, not occupied by a wall. In other words, we have found an exit from the maze.

4. We have explored a square unsuccessfully in all four directions.

Now we will considering the following maze:



The `Maze` object provides the following methods for our search algorithm:

1. `Maze(filename, useGui)` reads a maze and finds its starting position. Set `useGui` to `true` to open the drawing window; omitting it selects text-only mode.

2. `print()` prints the current character grid.

The `Maze` object provides the following methods for our search algorithm:

1. `Maze(filename, useGui)` reads a maze and finds its starting position. Set `useGui` to `true` to open the drawing window; omitting it selects text-only mode.

2. `print()` prints the current character grid.

3. `updatePosition()` updates a grid square and its colored marker when GUI mode is enabled.

4. `isExit()` checks whether the current position is an exit.

5. `waitForClose()` keeps the completed maze visible until you close its window.

The `Maze` object provides the following methods for our search algorithm:

1. `Maze(filename, useGui)` reads a maze and finds its starting position. Set `useGui` to `true` to open the drawing window; omitting it selects text-only mode.

2. `print()` prints the current character grid.

3. `updatePosition()` updates a grid square and its colored marker when GUI mode is enabled.

4. `isExit()` checks whether the current position is an exit.

5. `waitForClose()` keeps the completed maze visible until you close its window.

The `Maze` class also provides `get(row, col)` so that our algorithm can easily access the status of any particular square.

We represent each square with a named constant. The GUI marks tried squares in blue, dead ends in red, and the successful path in green:

```
const char START = 'S';
const char OBSTACLE = '+';
const char TRIED = '.';
const char DEAD_END = '-';
const char PART_OF_PATH = 'O';
```

We represent each square with a named constant. The GUI marks tried squares in blue, dead ends in red, and the successful path in green:

```
const char START = 'S';
const char OBSTACLE = '+';
const char TRIED = '.';
const char DEAD_END = '-';
const char PART_OF_PATH = 'O';
```

The constructor reads a maze file: `+` represents a wall, a space represents an open square, and `S` marks the start. It checks the local file first; the course kernel also includes `maze2.txt` as a fallback. The following excerpts show the core grid-loading logic; validation, fallback lookup, and GUI setup are omitted.

```
class Maze {
    public:
        Maze(string mazeFilename, bool useGui = false) {
            ifstream mazeFile(mazeFilename);
            string line;
            while (getline(mazeFile, line)) {
                mazeList.push_back(line);
            }
            rowsInMaze = mazeList.size();
            columnsInMaze = mazeList[0].size();
            ...
```

```
            ...
            for (int row = 0; row < rowsInMaze; row++) {
                size_t col = mazeList[row].find(START);
                if (col != string::npos) {
                    startRow = row;
                    startCol = col;
                    break;
                }
            }
        }

        int startRow;
        int startCol;
    private:
        vector<string> mazeList;
        int rowsInMaze;
        int columnsInMaze;
};
```

Each call to `updatePosition()` changes the character grid. In GUI mode, it also moves the turtle and updates the colored marker, making exploration and backtracking visible.

```
void updatePosition(int row, int col, char val) {
    mazeList[row][col] = val;
    if (gui) gui->update(row, col, val);
}

void print() {
    for (string& row : mazeList) {
        cout << row << endl;
    }
}
```

```
bool isExit(int row, int col) {
    return (row == 0 || row == rowsInMaze - 1 ||
            col == 0 || col == columnsInMaze - 1);
}

char get(int row, int col) {
    return mazeList[row][col];
}
```

```
bool searchFrom(Maze& maze, int row, int column) {
    // Base Case return values:
    //  1. We have run into an obstacle, return false
    if (maze.get(row, column) == OBSTACLE) {
        return false;
    }
    //  2. We have found an already explored square
    if (maze.get(row, column) == TRIED || maze.get(row, column) == DEAD_END)
{
        return false;
    }
    //  3. We have found an exit
    if (maze.isExit(row, column)) {
        maze.updatePosition(row, column, PART_OF_PATH);
        return true;
    }
    maze.updatePosition(row, column, TRIED);
    ...
```

```
    ...
    // Otherwise, use logical short circuiting to try each direction
    bool found = searchFrom(maze, row - 1, column)
              || searchFrom(maze, row + 1, column)
              || searchFrom(maze, row, column - 1)
              || searchFrom(maze, row, column + 1);
    if (found) {
        maze.updatePosition(row, column, PART_OF_PATH);
    } else {
        maze.updatePosition(row, column, DEAD_END);
    }
    return found;
}
```

(complete listing: `pythonds3/cppds/maze.hpp`)

You will notice that in the recursive step there are four recursive calls to `searchFrom()`. It is hard to predict how many of these will be used since they are connected by `||` — if the first call succeeds, short-circuit evaluation skips the rest.

You will notice that in the recursive step there are four recursive calls to `searchFrom()`. It is hard to predict how many of these will be used since they are connected by `||` — if the first call succeeds, short-circuit evaluation skips the rest.

If the first call to `searchFrom()` returns `true`, short-circuit evaluation skips the other three calls. Geometrically, the north step is then on a path leading out of the maze.

You will notice that in the recursive step there are four recursive calls to `searchFrom()`. It is hard to predict how many of these will be used since they are connected by `||` — if the first call succeeds, short-circuit evaluation skips the rest.

If the first call to `searchFrom()` returns `true`, short-circuit evaluation skips the other three calls. Geometrically, the north step is then on a path leading out of the maze.

If north fails, the next recursive call tries south, then west, then east. If all four calls return `false`, the current square is a dead end.

In [ ]:

```cpp
#include <iostream>
#include "pythonds3/cppds/maze.hpp"   // Maze class + searchFrom (md listing above)
using namespace std;

int main() {
    Maze myMaze("maze2.txt", true);
    searchFrom(myMaze, myMaze.startRow, myMaze.startCol);
    myMaze.print();   // O = path, . = tried, - = dead end
    myMaze.waitForClose();
    return 0;
}
```

The sample `maze2.txt` is included with the course kernel. Put your own `maze2.txt` beside the notebook to use it instead. The included sample is:

```
++++++++++++++++++++++
+   +   ++ ++        +
      +     ++++++++++
+ +    ++  ++++ +++ ++
+ +   + + ++    +++  +
+          ++  ++  + +
+++++ + +      ++  + +
+++++ +++  + +  ++   +
+          + + S+ +  +
+++++ +  + + +     + +
++++++++++++++++++++++
```

Run the example with the course **C++ 17** kernel. Orange squares are walls; blue marks exploration, red marks dead ends, and green marks the successful path. Close the maze window to finish the cell.

Change `true` to `false` for text-only execution: `print()` still shows the final grid, and `waitForClose()` returns immediately.

In [15]:
```
from jupytercards import display_flashcards
fpath = "https://raw.githubusercontent.com/phonchi/nsysu-
math208/refs/heads/main/extra/flashcards/"
display_flashcards(fpath + "ch6.json")
```

Recursion



Next

>

References

1. Textbook: *Problem Solving with Algorithms and Data Structures using C++* (cppds), Chapter 5 (Recursion) — https://runestone.academy/ns/books/published/cppds/index.html